

CSCI-UA 102 — Quiz 10

Selection / Quickselect

Name: _____ NetID: _____

(2 points) Write a method `Integer[][] splitByPivot(Integer[] arr, Integer pivot)` that returns two arrays: one containing all the elements of `arr` that are smaller than `pivot`, and one containing the rest.

(1 point) How can you tell which array contains the L -th largest element of `arr`?

(2 points) Say I give you a method `Integer pivotPicker(Integer[] arr)` that picks a pivot to split an array. Write a method that uses `splitByPivot` and `pivotPicker` to write `Integer median(Integer[] arr)` which finds the median of `arr`. You may assume `arr` has an odd number of elements.

Hint: write a helper method `Integer getRank(Integer[] arr, int rank)` that gives back the `rank`-th largest element in `arr`. Then recurse using this helper.

Bonus (1 point) Say the pivot was always between the first and third quartile of the elements in `arr`. What is the worst-case asymptotic computational complexity of `median`? Write out your argument!

Solution

Part 1: splitByPivot.

```
Integer[][] splitByPivot(Integer[] arr, Integer pivot) {
    int smallerCount = 0;
    for (Integer x : arr)
        if (x < pivot) smallerCount++;

    Integer[] smaller = new Integer[smallerCount];
    Integer[] rest     = new Integer[arr.length - smallerCount];

    int si = 0, ri = 0;
    for (Integer x : arr) {
        if (x < pivot) smaller[si++] = x;
        else          rest[ri++]    = x;
    }
    return new Integer[][]{smaller, rest};
}
```

Part 2: which array contains the L -th largest?

The `rest` array contains all elements $\geq \text{pivot}$ — these are the larger elements. So:

- If $L \leq |\text{rest}|$: the L -th largest is in `rest`.
- Otherwise: the L -th largest is in `smaller`, with adjusted rank $L - |\text{rest}|$.

Part 3: median.

For an array of n elements (odd), the median is the $\lceil n/2 \rceil$ -th largest.

```
Integer median(Integer[] arr) {
    return getRank(arr, arr.length / 2 + 1);
}

Integer getRank(Integer[] arr, int rank) {
    if (arr.length == 1) return arr[0];

    Integer pivot = pivotPicker(arr);
    Integer[][] split = splitByPivot(arr, pivot);
    Integer[] smaller = split[0];
    Integer[] rest     = split[1];

    if (rank <= rest.length)
        return getRank(rest, rank);
    else
        return getRank(smaller, rank - rest.length);
}
```

Bonus: complexity.

If the pivot is always between Q_1 and Q_3 , then both `smaller` and `rest` have size between $n/4$ and $3n/4$. So at each recursive call, the array we recurse on has size at most $3n/4$.

The work at each level (computing `splitByPivot`) is $O(n)$. Letting $T(n)$ denote the worst-case time:

$$T(n) \leq T(3n/4) + cn$$

for some constant c . Unrolling the recurrence:

$$T(n) \leq cn + c(3n/4) + c(3n/4)^2 + \dots$$

This is a geometric series with ratio $3/4 < 1$, so it sums to:

$$T(n) \leq \frac{cn}{1 - 3/4} = 4cn = O(n)$$

Therefore `median` runs in $O(n)$ in the worst case under this assumption.

Intuition: we throw away at least a quarter of the array at every step, so the array shrinks geometrically. The total work is dominated by the first call ($O(n)$), and the rest is a geometric series that converges.